



CLINICAL TRIALS DATABASE:

DATABASES, GIBM – 25/26

ALEJANDRO RAMOS, MENCÍA GARCÍA, LOUISE STANDAERT,
ÁLVARO MÉNDEZ

1. Introduction

2. General Structure of the System

2.1 Pojos Package → Título 2

2.2 JDBC Package → Título 2

2.3 JPA Package → Título 2

2.4 XML Package → Título 2

2.5 Main Package → Título 2

3. Database Design

3.1 Patients

3.2 Clinical Description

3.3 Trial

3.4 Hospital

3.5 Doctor

4. Explanation of the Relationships

4.1 Patients and Clinical Description

4.2 Patients and Trial

4.3 Hospital and Patients

4.4 Hospital and Doctor

4.5 Doctor and Trial

4.6 Hospital and Trial

5. Java Class Design

5.1 POJO Classes

5.2 Manager Classes

5.3 ConnectionManager

5.4 JPA and Login System

5.5 XML System

6. Database Integrity

7. Design Decisions

8. Conclusion

1. Introduction

The aim of this project is to design and implement a database system for managing clinical trials. The system stores information about patients, doctors, hospitals, trials and clinical descriptions. It also includes user authentication, role management and XML import/export functionality.

The design is based on an Entity-Relationship model and later translated into Java classes, database tables and manager classes. The ER diagram defines the main entities of the system: Patients, Clinical Description, Trial, Hospital and Doctor. It also represents how these entities are connected through relationships such as patients having descriptions, patients being assigned to trials, hospitals hosting trials, hospitals hiring doctors and doctors supervising trials.

The main objective of the design was to create a system that is organized, modular and easy to maintain. For this reason, the project separates data classes, database access classes, authentication classes and XML management classes.

2. General System Design

The project is divided into several packages:

- Pojos
- jdbc
- jpa
- xml
- main

This separation was made to keep the system organized. Each package has a specific responsibility.

The Pojos package contains the basic Java classes that represent the entities of the database. These classes include Patients, Trial, Doctors, Hospitals, Description, HospitalTrial, User and Role.

The jdbc package contains the manager classes responsible for communicating with the relational database. These managers execute SQL queries and provide methods to insert, remove, update and search information.

The jpa package is used for user and role management. It includes authentication functionality and password hashing.

The xml package allows the system to export and import data using XML files.

The main package contains the main execution logic of the program, including the console menu and the graphical interface.

This structure was chosen because it follows a layered design. Each part of the system has a clear purpose, which makes the project easier to understand, test and modify.

3. Entity-Relationship Design

The ER diagram represents the conceptual structure of the system. The main entities are:

- **Patients**

The Patients entity stores the information about each patient. It includes attributes such as:

- patientID
- patientName
- results
- hospitalID
- trialID
- descriptionID

These attributes were chosen because a patient needs to be identified, linked to a hospital, optionally assigned to a clinical trial and connected to a clinical description. The results attribute is used to store the patient test result, such as positive or negative.

Patients are central in the system because clinical trials depend on patient participation.

- **Clinical Description**

The Clinical Description entity stores additional medical information about a patient. It includes:

- descriptionID
- gender
- cause
- patientID

This entity was separated from Patients because clinical description data represents medical details rather than general patient identification data. Keeping it in a separate entity improves organization and avoids making the patient table too large.

The relationship between Patients and Clinical Description is one-to-one. Each patient has one clinical description, and each clinical description belongs to one patient. This is why the system uses a descriptionID and a patientID to connect both entities.

- **Trial**

The Trial entity stores the information about each clinical trial. Its attributes are:

- trialID
- trialName
- durationDay
- budget
- targetPatient
- startingDate

These attributes were chosen because they describe the basic administrative and financial information of a clinical trial. The durationDay attribute stores how long the trial lasts, budget represents the financial cost and targetPatient indicates the number of patients required.

A trial can include several patients, so the relationship between Trial and Patients is one-to-many. One trial can have many patients, but each patient is assigned to one trial at a time.

- **Hospital**

The Hospital entity represents the hospitals participating in the system. It includes:

- hospitalID
- hospitalName
- hospitalCity

These attributes were chosen because they identify each hospital and provide its location. Hospitals are important because they can host trials, hire doctors and have patients.

The relationship between Hospital and Patients is one-to-many. One hospital can have many patients, but each patient belongs to one hospital.

- **Doctor**

The Doctor entity stores information about medical staff. It includes:

- doctorID
- name
- hospitalID
- trialID

- gender
- speciality

Doctors are linked to hospitals and trials. A hospital can hire many doctors, but each doctor belongs to one hospital. A trial can also be supervised by doctors, so doctors are connected to trials through the trialID.

The speciality attribute was implemented using an enum called DoctorSpecialty. This was done to avoid invalid or inconsistent specialty names. For example, instead of allowing the user to type any text, the system limits the possible values to predefined options such as cardiology, oncology, neurology, pediatrics and general medicine.

4. Relationship Design

Patients and Clinical Description

The relationship between Patients and Clinical Description is one-to-one.

This design was chosen because each patient should have one specific clinical description. The description contains medical information related only to that patient.

In the database, this is implemented using descriptionID and patientID.

Patients and Trial

The relationship between Patients and Trial is many-to-one from the patient side.

One trial can contain many patients, but each patient is assigned to one trial. This design makes sense because a clinical trial usually requires multiple participants.

In the Java system, this relationship is managed by methods such as enrolling a patient into a trial and removing a patient from a trial.

Hospital and Patients

The relationship between Hospital and Patients is one-to-many.

One hospital can manage many patients, but each patient belongs to one hospital. This allows the system to know where each patient is being treated.

Hospital and Doctor

The relationship between Hospital and Doctor is one-to-many.

One hospital can hire many doctors, while each doctor is assigned to one hospital. This was implemented by including hospitalID in the Doctors class and in the doctor table.

Doctor and Trial

The relationship between Doctor and Trial allows doctors to supervise trials.

This relationship is important because every clinical trial should be supervised by medical staff. The trialID attribute in the doctor entity allows the system to identify which trial a doctor is supervising.

Hospital and Trial

The relationship between Hospital and Trial is many-to-many.

A hospital can host several trials, and the same trial can be carried out in several hospitals. To solve this many-to-many relationship, we created the intermediate class HospitalTrial.

This is why the UML contains the class:

HospitalTrial

with the attributes:

hospitalId

trialId

This intermediate class represents the relational table that connects hospitals and trials. This design avoids duplicated data and follows relational database normalization principles.

5. UML Design

The UML class diagram represents the structure of the Java application.

The system is divided into different types of classes:

POJO Classes

The POJO classes represent the data entities. They contain attributes, constructors, getters, setters, equals, hashCode and toString methods.

Examples:

Patients

Trial

Doctors

Hospitals

Description

HospitalTrial

User

Role

These classes were designed to store data in a simple way. They do not contain complex database logic. This makes the code cleaner because the entity classes only represent information.

Manager Classes

The manager classes are responsible for database operations. Examples include:

PatientManager
TrialManager
DoctorManager
HospitalManager
DescriptionManager
HospitalTrialManager

Each manager controls the operations related to one part of the system. For example, PatientManager manages patient insertion, deletion and searches. TrialManager manages trial creation, trial removal, patient enrollment and calculations such as average budget or average duration.

This design was chosen to avoid placing SQL queries inside the Main class. If all SQL queries were written in Main, the program would become difficult to read and maintain. By using manager classes, each responsibility is separated.

ConnectionManager

The ConnectionManager class controls the connection to the SQLite database.

It includes methods such as:

getConnection()
closeConnection()
createTables()
initializeData()

This class was created to centralize database connection management. Instead of opening a new connection manually in every class, the project uses ConnectionManager to control the connection in a consistent way.

JPA Classes

The JPA_manager class is used for managing users and roles. It includes methods for creating users, creating roles, logging in, checking login information and updating passwords.

The PasswordUtil class hashes and verifies passwords. This was included for security reasons. Storing plain text passwords would be unsafe, so the system hashes passwords before saving them.

The User and Role classes represent the authentication model. A user has one role, and a role can be assigned to several users. This allows the system to distinguish between different types of users, such as administrators, doctors or guests.

XML Classes

The XML part of the project includes:

ClinicalTrialsXMLDataBase
XmlManager
LocalDateAdapter

The ClinicalTrialsXMLDataBase class groups lists of doctors, hospitals, patients, descriptions, trials and hospital-trial relationships.

The XmlManager class is responsible for marshalling and unmarshalling data. This means that the system can convert Java objects into XML files and also read XML files back into Java objects.

The LocalDateAdapter class was necessary because LocalDate needs special handling when being converted to and from XML.

This XML functionality was added to provide data portability. It allows the information of the system to be exported, stored and imported again.

6. Database Integrity

The system uses primary keys and foreign keys to maintain data integrity.

Primary keys are used to uniquely identify each record. For example:

patientID
trialID
hospitalID
doctorID
descriptionID

Foreign keys are used to connect tables. For example, a patient stores trialID, hospitalID and descriptionID because the patient is related to a trial, a hospital and a clinical description.

This design prevents invalid data. For example, a patient should not be assigned to a trial that does not exist. If the user tries to insert a patient with an invalid trialID, the database raises a foreign key error. This is why the system should check that related IDs exist before inserting new records.

7. Why This Design Was Chosen

This design was chosen for several reasons.

First, the system follows the ER diagram closely. Each main entity in the ER diagram was converted into a Java class and a database table. This makes the implementation consistent with the conceptual design.

Second, the project separates responsibilities. POJO classes store data, manager classes access the database, JPA classes manage users, and XML classes manage data export and import. This makes the code easier to maintain.

Third, the many-to-many relationship between hospitals and trials was solved using an intermediate class, HospitalTrial. This is the correct relational database solution because it avoids duplicated information and allows a hospital to host many trials while also allowing a trial to be hosted by many hospitals.

Fourth, the use of enums for doctor specialties improves data consistency. Instead of allowing any text value, the program restricts the specialty to a predefined list.

Fifth, password hashing was included to improve security. The system does not depend only on a simple username and password comparison; it uses a utility class to hash and verify passwords.

Finally, XML support was added to make the system more flexible. The database information can be exported and imported, which is useful for backups or data exchange.

8. Conclusion

In conclusion, the system was designed to manage clinical trials in an organized and structured way. The ER diagram defines the database model, while the UML diagram represents the Java implementation.

The design separates the project into clear layers: data entities, database managers, authentication and XML management. This makes the system easier to understand and extend.

The most important design decision was to represent the real relationships between patients, trials, hospitals, doctors and descriptions correctly. The use of foreign keys, manager classes, POJOs, JPA authentication and XML export creates a complete system that is suitable for managing clinical trial information.

